Лабораторна робота № 8

Тема. Багатопотокове та багатопроцесне програмування. Асинхронна взаємодія, мережеві запити та робота з SQLite.

Мета: ознайомитися з принципами багатопотокового та багатопроцесного програмування, навчитися використовувати асинхронну взаємодію для виконання задач, виконувати мережеві запити та отримувати практичні навички роботи з базою даних SQLite.

Короткі теоретичні відомості

Багатопотокове програмування – це спосіб організації обчислювального процесу, за якого в межах одного процесу операційної системи створюється та виконується кілька потоків.

Потік (thread) – найменша одиниця виконання в межах одного процесу. Кілька потоків у межах одного процесу виконуються конкурентно, тобто ніби одночасно, але фактично перемикаючись між собою, поділяючи спільну пам’ять. Модуль *threading* надає низькорівневий інтерфейс для створення та управління потоками. Приклад:

```
import threading
import time

def worker(name, delay):
    print(f"Потік {name} почав роботу")
    time.sleep(delay) # імітація I/O-операції
    print(f"Потік {name} завершення роботи")

# створення потоків
threads = []
for i in range(3):
    threads.append(threading.Thread(target=worker,
args=(f"Thread-{i}", 2)))

# Запуск всіх потоків
for t in threads:
    t.start()
# Очікування завершення всіх потоків
for t in threads:
    t.join()
print("Всі потоки завершено")
```

Оскільки потоки мають спільну пам’ять, доступ до спільних даних потрібно синхронізувати, щоб уникнути гонки даних (race condition). Приклад синхронізації потоків за допомогою *Lock*:

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter # працюємо з глобальною змінною
    for _ in range(100000):
        with lock: # блокуємо доступ до спільного ресурсу
            counter += 1

threads = [threading.Thread(target=increment) for _ in range(10)]

for t in threads:
    t.start()

for t in threads:
    t.join()

print(f"Counter: {counter}") # Вивід: 1000000
```

Багатопроцесне програмування – це спосіб організації обчислювального процесу, за якого програма створює кілька незалежних процесів операційної системи.

Процес – незалежний екземпляр програми з власним інтерпретатором Python, власною пам’яттю та власним GIL. Кілька процесів можуть виконуватися паралельно на різних ядрах процесора.

GIL (Global Interpreter Lock) – це механізм у CPython (стандартному інтерпретаторі), який дозволяє лише одному потоку виконувати код у будь-який момент часу.

Для роботи з процесам використовується модуль *multiprocessing*, який має API подібний до *threading*. Приклад:

```
from multiprocessing import Process
import os
```

```
def worker(name):
    print(f"Процес {name}, PID: {os.getpid()}")

if __name__ == "__main__": # перевірка чи запущено файл безпосередньо
    processes = []
    for i in range(4):
        p = Process(target=worker, args=(f"Process-{i}",))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

    print("Всі процеси завершено")
```

Pool дозволяє розподілити виконання функції між кількома процесами. Приклад використання пулу процесів:

```
from multiprocessing import Pool

def square(x):
    return x * x

if __name__ == "__main__":
    with Pool(4) as pool:
        results = pool.map(square, [1, 2, 3, 4, 5, 6, 7])
        print(results) # Вивід: [1, 4, 9, 16, 25, 36, 49]
```

Модуль *concurrent.futures* надає ще вищий рівень абстракції з однаковим інтерфейсом для потоків та процесів. Приклад:

```
from concurrent.futures import ThreadPoolExecutor,
ProcessPoolExecutor
import time

def cpu_heavy(n):
    """Обчислювальна задача"""
    total = 0
    for i in range(n):
        total += i ** 2
    return total
```

```
def io_bound(url):
    """Імітація I/O-орієнтованої задачі"""
    time.sleep(1) # імітація мережевого запиту
    return f"Завершено: {url}"

# Використано потоки для I/O-задач
with ThreadPoolExecutor(max_workers=4) as executor:
    print(list(executor.map(io_bound, ["url1", "url2", "url3",
"url4"])))

# Для CPU-задач використано процеси
if __name__ == "__main__":
    with ProcessPoolExecutor(max_workers=4) as executor:
        print(list(executor.map(cpu_heavy, [10**6] * 4)))
```

Асинхронна взаємодія (асинхронне програмування) – це парадигма написання конкурентного коду, за якої кілька завдань виконуються в межах одного потоку (одного процесу) без блокування виконання програми під час очікування на тривалі операції. Цей підхід базується на концепції кооперативної багатозадачності: завдання (корутини) самостійно «віддають» управління головному диспетчеру (циклу обробки подій / event loop) в момент очікування за допомогою ключового слова *await*. Основою асинхронності є event loop, який постійно моніторить стан усіх виконуваних операцій і, щойно якась операція завершилася, передає управління відповідній корутині для продовження роботи.

Корутина (Coroutine) – це спеціальна функція, визначена за допомогою *async def*. Вона може призупинити своє виконання ключовим словом await і відновлюватися згодом.

await – оператор, який вказує, що програма має почекати завершення асинхронної операції, але при цьому може переключитися на інші завдання.

Future/Task – обгортка для корутини, яка представляє результат операції, що виконується асинхронно. Дозволяє керувати станом виконання.

Приклад використання модуля *asyncio* для реалізації асинхронної взаємодії:

```
import asyncio

async def fetch_data(url):
    print(f"Починається завантаження {url}")
    await asyncio.sleep(2) # імітація очікування мережевої відповіді
    print(f"Завершено завантаження {url}")
    return f"Дані з {url}"

async def main():
    # Запуск кількох корутин одночасно
    task1 = asyncio.create_task(fetch_data("https://site1.com"))
    task2 = asyncio.create_task(fetch_data("https://site2.com"))

    results = await asyncio.gather(task1, task2)
    print(results)

# запуск event loop
asyncio.run(main())
```

Для виконання мережевих запитів у Python використовується модуль *urllib*. Приклад використання urllib для виконання GET- і POST-запитів.

```
from urllib.request import urlopen, Request
import json

with urlopen("https://jsonplaceholder.typicode.com/posts/1") as
response: # GET-запит
    print(json.load(response))

request = Request(
    "https://jsonplaceholder.typicode.com/posts/",
    data=json.dumps({"title": "Python", "body": "Example POST
request", "userId": 1}).encode("utf-8"),
    headers={"Content-Type": "application/json"},
    method="POST"
)

with urlopen(request) as response:
    print(json.load(response))
```

SQLite – це полегшена реляційна база даних, яка не потребує окремого сервера та зберігає всю інформацію у звичайному файлі на диску (зазвичай із розширенням *.db*). Робота з SQLite реалізована через вбудований модуль *sqlite3*. Приклад створення таблиці, додавання та зчитування записів:

```
import sqlite3

# Підключення до файлу (або створення нового)
with sqlite3.connect("my_database.db") as connection:
    # Створення курсора для виконання запитів
    cursor = connection.cursor()

    # Створення таблиці
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER
    )
    """)

    # Додавання одного запису
    cursor.execute("""
    INSERT INTO users (name, age)
    VALUES (?, ?)
    """, ("Олена", 20))

    # Отримання всіх записів
    cursor.execute("SELECT * FROM users")
    for user in cursor.fetchall():
        print(user)

    # При успішному завершенні блоку with
    # зміни автоматично фіксуються.
    # connection.commit()
```

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.

2. Виконати HTTP-запити в окремих потоках до обраного REST API (наприклад, JSONPlaceholder): отримати дані за допомогою GET-запиту та

створити новий запис за допомогою POST-запит (якщо доступний). Отриману інформацію зберегти у базі даних SQLite, попередньо створивши таблицю для записів. Також реалізувати зчитування збережених даних.

3. Написати звіт з виконання лабораторної роботи.

Контрольні питання

1. Що таке GIL?
2. Який модуль використовується для створення та управління потоками?
3. У яких випадках доцільно використовувати багатопотоковість, а в яких – багатопроцесність?
4. Як називається центральний механізм, який керує виконанням корутин в асинхронному програмуванні?
5. Який модуль дозволяє виконувати HTTP-запити?
6. Який метод використовується для фіксації змін у базі даних SQLite після виконання INSERT, UPDATE, DELETE?